J26-SE-347

Secure Multi-Tenant Natural Language Interface for Database Querying

IT4010 Research Project – 2026 Jul

Component Development Plan

V1.0

1. Component details

Member	Navodya P G D
Registration No	IT23144330
Component	3. AI Query Security & Validation Layer
Sub-objective (from TAF)	To design an AST-driven security middleware that mathematically validates generated queries, prevents prompt-injected semantic attacks, and securely orchestrates preview-based controlled write transactions.
Novelty (from TAF)	A defense-in-depth LLM-database gateway combining pre-execution AST allowlisting, dynamic in-pipeline PII masking, and a novel transaction-rollback preview mechanism for governing exceptional multi-tenant writes.
Code location	`security_validation_layer_js/` (Node.js, node-sql-parser, Express)

2. Current progress against TAF tasks

TAF task	Status	What exists now
AST parsing engine to validate query safety and block destructive SQL	**Mostly done**	• SELECT-only by default; DROP/TRUNCATE/ALTER/GRANT etc. always blocked • Stacked-statement, tautology, UNION, comment and subquery-depth checks • Dangerous-function blocklist (JSON config) • Express `securityMiddleware()` + demo UI • First manual test pass (xlsx report)
Controlled Write Protocol (transactional preview + rollback)	**Stub only**	`previewWrite()`, `WriteApprovalRequest` placeholders
PII detection (NER + regex)	**Stub only**	`mask()` placeholder
Format-preserving encryption and dynamic field masking	**Not started**	—
Re-validation after Member 3's tenant-isolation rewrite	**Stub only**	`validateRewritten()` placeholder

3. Improvements to the completed validation engine

The following gaps were found by running edge-case queries through `validate()` on 23 September 2026. Each one is a concrete fix for the existing engine, listed in priority order.

#	Gap found	Evidence (current result)	Planned fix	Priority
1	No table/column allowlist; only statement types are allowlisted	`SELECT * FROM information_schema.tables` → SAFE `SELECT user, authentication_string FROM mysql.user` → SAFE	Per-tenant schema allowlist of tables and columns; always block information_schema, mysql, performance_schema, sys. Add `getColumns()`.	High
2	System variables not blocked	`SELECT @@version` → SAFE	Reject any @@ system-variable node in the AST.	High

#	Gap found	Evidence (current result)	Planned fix	Priority
3	Tautology check only catches literal = literal	`WHERE id = 5 OR 1, OR 2 > 1, OR id = id` → SAFE	Constant-fold expressions; flag always-true OR branches and self-comparisons.	High
4	Comment check gives false positives on string literals	`WHERE note = 'item #3'` → BLOCKED	Strip string literals before scanning for `--`, `#`, `/*`.	Medium
5	No resource / locking limits	`SELECT * FROM orders FOR UPDATE`, `LIMIT 100000000`, 4-way cross join → SAFE	Block FOR UPDATE / LOCK IN SHARE MODE; enforce max LIMIT; cap joins; EXPLAIN-based row estimate before execution.	Medium
6	Blocklist missing error-based injection functions	—	Add EXTRACTVALUE, UPDATEXML and user-defined function calls; block SET statements.	Medium
7	Testing is manual (xlsx)	—	Jest test suite: one test per rule, attack corpus and safe-query corpus, run with `npm test`.	High
8	No audit trail	—	Structured JSON audit log per decision (tenant, user, SQL hash, violations, latency).	Medium
9	Violations lack severity / stable codes	—	Add severity and error codes so Member 1's refinement loop can decide whether to regenerate.	Low

4. Remaining components to develop (next steps)

Component	Tasks	Deliverable
A. Re-validation after rewrite `validateRewritten()`	• Agree the AST hand-off contract with Member 3 (input: AST + tenant context; output: rewritten AST) • Re-run every validation rule on the rewritten AST • Verify a `tenant_id = <JWT value>` predicate exists on every table, including subqueries and JOINs	Working function + tests; this makes dual-stage enforcement verifiable
B. Controlled Write Protocol	• MySQL (mysql2): START TRANSACTION → capture before-image → execute → capture after-image and affected rows → ROLLBACK • State machine: pending → previewed → approved → committed | rejected | expired • Bind approval to a hash of the SQL; refuse commit if SQL changed • Approval expiry (TTL); re-check affected-row count at commit • Max affected rows per write; approver must have a different role from requester • Endpoints: POST /writes/preview, /writes/:id/approve, /writes/:id/commit	Preview-and-rollback write workflow on a Docker MySQL with synthetic data

Component	Tasks	Deliverable
C. PII detection & masking	• Layer 1: PII tags in schema metadata (e.g. customers.email → EMAIL) • Layer 2: regex + validators: email, phone, Sri Lankan NIC (old/new), card numbers (Luhn), IBAN • Layer 3: NER on free-text columns using Microsoft Presidio (Python/spaCy) as a side service • Role-based masking policy: full / partial / hashed / format-preserving encrypted • Format-preserving encryption with NIST FF1 (SP 800-38G) • Track PII lineage through the AST (GROUP BY email, CONCAT(name, …))	Implemented `mask()` in the result pipeline before Member 4's layer

5. Integration contract with team members

Member	What I receive	What I provide
Sasmitha A W D (IT23143418) NL-to-SQL Engine	Raw SQL + request/tenant context	ValidationResult with violation codes and severity, for the self-correction loop
Jinajith R A B (IT23212268) Access Control	Tenant context from JWT, roles, rewritten AST	Parsed AST and helper functions; re-validation of the rewritten AST; role checks for write approval
Wijethunge W M V (IT23256750) Visualization & Prediction	—	Masked result rows plus column metadata (which fields were masked and how)

Action: write a short shared interface document (input/output JSON, tenant-context format, violation codes) and agree it with the team before building `validateRewritten()`.

6. Evaluation plan

Metric	Method
False-positive rate	Run the validator over the gold SELECT queries in the Spider and BIRD benchmarks; almost all should pass.
Attack detection rate	Attack corpus (OWASP cases, sqlmap payloads, prompt-injection-generated SQL) compared against a baseline regex/keyword filter.
Latency overhead	Added time per query, reported at p50 and p95.
PII detection	Precision and recall on synthetic Sales/HR/Finance data generated with Faker.
Write protocol integrity	Show that tampered, changed or expired write approvals are refused at commit time.

7. Proposed timeline

Weeks	Dates (2026)	Work	Output
1–2	23 Sep – 6 Oct	Jest tests + corpora; fixes #1–#4 from Section 3	Hardened validator with automated tests
3	7 Oct – 13 Oct	Interface contract with team; validateRewritten(); fixes #5–#9	Agreed interface document; re-validation working
4–7	14 Oct – 10 Nov	Controlled Write Protocol on Docker MySQL	Preview / approve / commit workflow
8–11	11 Nov – 8 Dec	PII pipeline: schema tags → regex → Presidio → masking → FF1	Working mask() in the result pipeline
12–13	9 Dec – 22 Dec	Integration with other components; evaluation runs	End-to-end pipeline; evaluation results for the report

Dates are a proposal and should be aligned with the IT4010 milestone schedule and supervisor feedback.